

SHORT PDF

TASK-4

1. MOORE FINITE STATE MACHINE

Theory

A Moore Machine is a type of Finite State Machine in which the output depends only on the present state of the system. The output does not change immediately with changes in input; instead, it changes only after the machine transitions into another state.

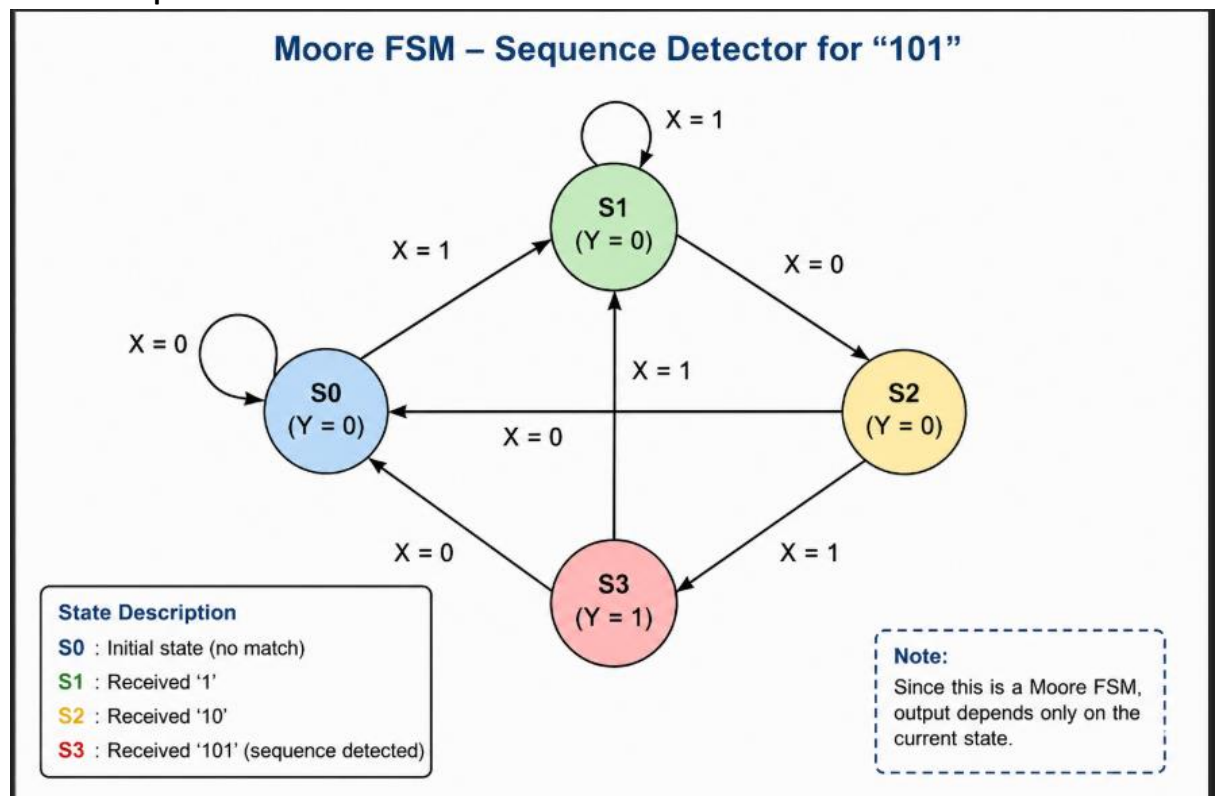
Mathematically,

Output = f(Present State)

Moore machines generally require more states than Mealy machines but provide more stable outputs because the outputs are synchronized with state transitions.

In this experiment, a Moore FSM was designed to detect the sequence **101**.

State Description



State	Meaning	Output Y
S0	Initial state	0
S1	Received '1'	0
S2	Received '10'	0
S3	Sequence '101' detected	1

The machine starts from S0. After receiving the sequence 101, it enters state S3 where the output becomes HIGH.

Design Logic

The design consists of three major blocks:

State Register

Stores the current state and updates it at the positive edge of the clock.

Next State Logic

Determines the next state depending on the current state and input.

Output Logic

Since it is a Moore machine, the output depends only on the current state.

The output becomes HIGH only when the FSM enters the detection state S3.

Simulation Results

The input sequence **101101** was applied.

The output became HIGH whenever the FSM entered state S3, indicating successful detection of the sequence 101.

The waveform showed that the output changed only after state transitions, demonstrating the fundamental characteristic of Moore machines.

Observation

The Moore FSM correctly detected the sequence 101. The output remained stable between clock transitions and depended exclusively on the present state. The simulated waveforms matched the theoretical state transitions and truth table.

2. MEALY FINITE STATE MACHINE

Theory

A Mealy Machine is a finite state machine in which the output depends on both the current state and the present input.

Mathematically,

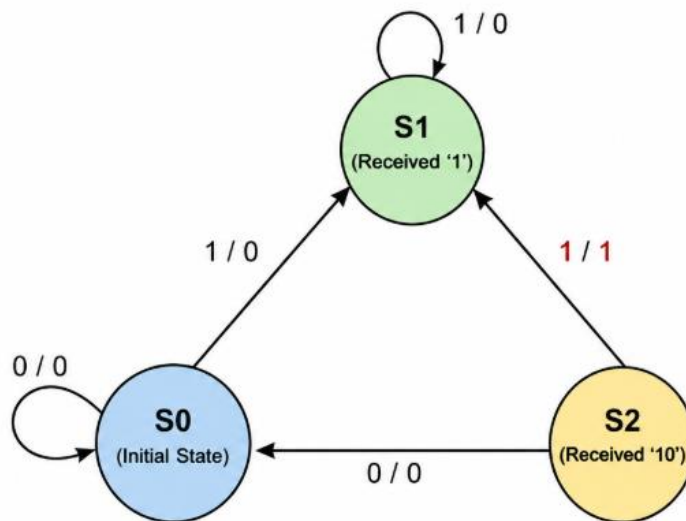
Output = f(Present State, Input)

Compared to Moore machines, Mealy machines require fewer states and produce faster outputs because the output is generated during state transitions.

A Mealy FSM was designed to detect the sequence **101**.

State Description

Mealy FSM – Sequence Detector for “101”



Note:

- This is a Mealy FSM. Output depends on Present State and Input.
- When the sequence '101' is detected (in state S2 and input X = 1), output Y = 1 is generated.

State Transitions (Mealy)

Present State	Input (X)	Next State	Output (Y)
S0	0	S0	0
	1	S1	0
S1	0	S2	0
	1	S1	0
S2	0	S0	0
	1	S1	1

State Description

S0 : Initial state (no match)

S1 : Received '1'

S2 : Received '10'

Output (Y) is shown on the transition as Input / Output.

Meaning

State

S0	Initial state
S1	Received '1'
S2	Received '10'

The sequence is detected when the machine is in S2 and receives input 1.

Design Logic

The FSM consists of:

State Register

Updates the present state at each positive clock edge.

Next State Logic

Determines the next state based on the current state and input.

Output Logic

The output is generated simultaneously with state transitions.

When the machine is in S2 and receives input 1, the sequence 101 is detected and the output immediately becomes HIGH.

Simulation Results

Input sequence:

101101

Output sequence:

001001

The output pulse occurred exactly at the instant the sequence was completed.

Observation

The Mealy FSM successfully detected the sequence 101 with fewer states than the Moore FSM. The output was generated immediately upon detection, illustrating the faster response characteristic of Mealy machines.

Comparison Between Moore and Mealy FSM

Parameter	Moore FSM	Mealy FSM
Output depends on	Present State	Present State and Input
Number of states	More	Fewer
Output response	Delayed by one state	Immediate
Stability	More stable	Faster response
Complexity	Higher	Lower

The comparison highlights the trade-off between output stability and response speed.

3. TRAFFIC LIGHT CONTROLLER

Theory

Traffic light controllers are practical applications of finite state machines. They ensure that traffic signals operate in a predefined sequence for efficient traffic management.

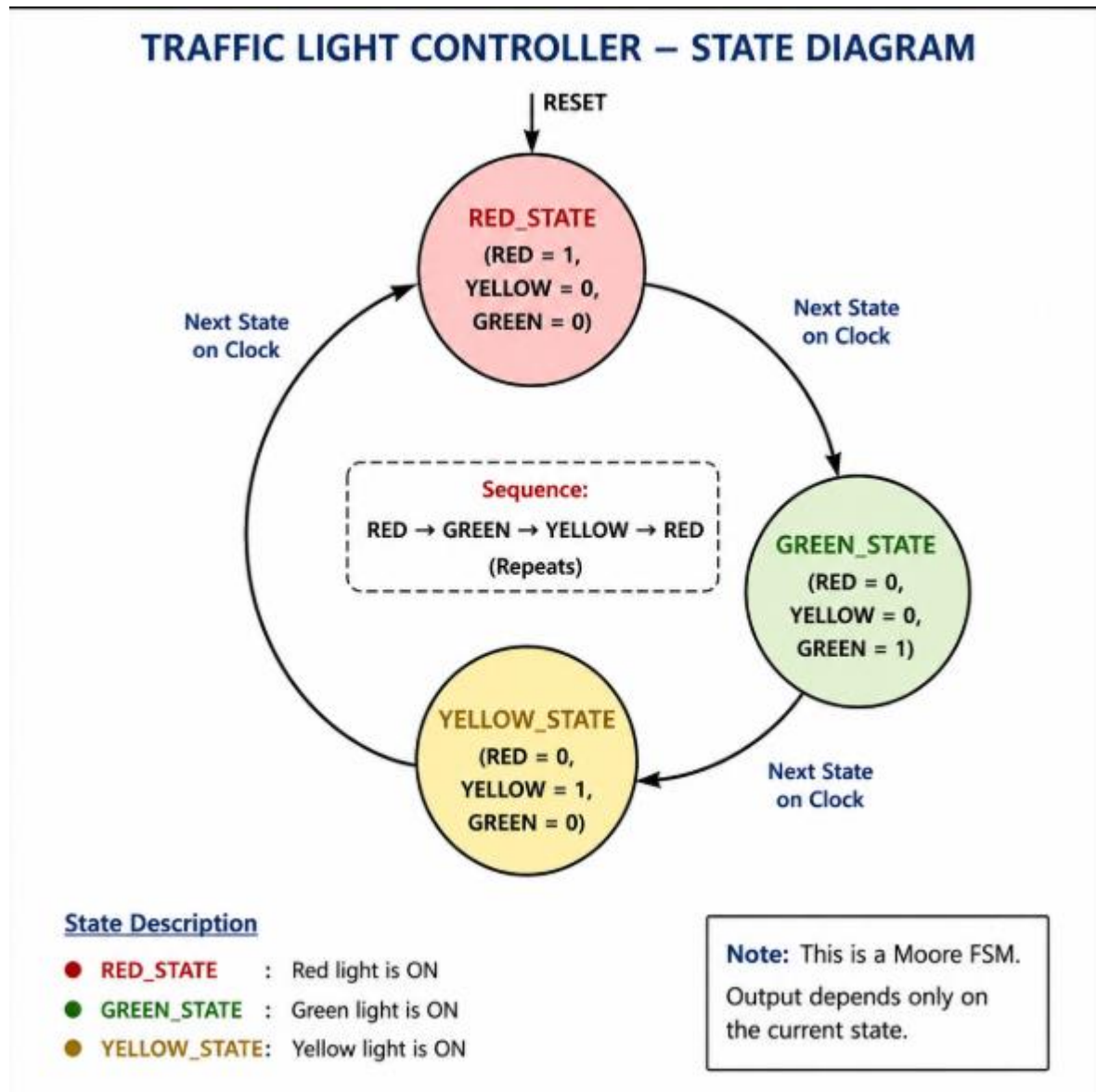
The controller implemented in this experiment operates using three states:

1. RED
2. GREEN
3. YELLOW

The sequence followed is:

RED → GREEN → YELLOW → RED

State Description



State	Red	Yellow	Green
RED	1	0	0
GREEN	0	0	1
YELLOW	0	1	0

Design Logic

The controller was implemented as a Moore FSM.
The current state determines which signal is active.
At every positive edge of the clock:

- RED transitions to GREEN.
- GREEN transitions to YELLOW.

- YELLOW transitions back to RED.
The RESET input initializes the system to the RED state.

Simulation Results

The simulated outputs showed the following sequence:

Clock Cycle	Active Signal
1	RED
2	GREEN
3	YELLOW
4	RED
5	GREEN
6	YELLOW

The waveform clearly demonstrated cyclic transitions among the three traffic signals.

Observation

The traffic light controller functioned correctly according to the predefined sequence. Only one light remained active at a time, ensuring safe operation. The simulation verified the practical implementation of FSM concepts in real-life applications.

4. SEQUENCE DETECTOR (MEALY FSM FOR 1101)

Theory

Sequence detectors are digital systems designed to recognize predefined bit patterns in serial data streams.

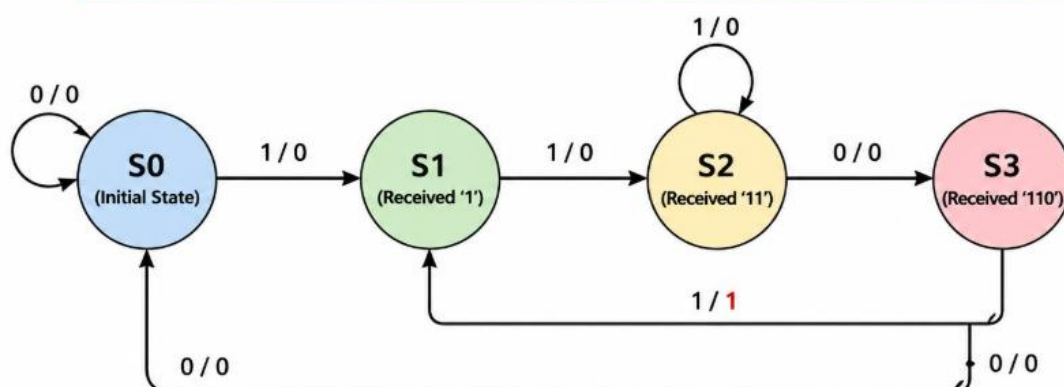
Sequence detectors find applications in:

- Digital communication systems
- Data synchronization
- Error detection
- Pattern recognition
- Embedded controllers

In this experiment, a Mealy FSM was designed to detect the sequence:
1101

State Description

SEQUENCE DETECTOR (Mealy FSM) FOR SEQUENCE "1101"



STATE DESCRIPTION

S0 (Blue)	: Initial state (no match)
S1 (Green)	: Received '1'
S2 (Yellow)	: Received '11'
S3 (Pink)	: Received '110'

Legend: X / Y → Input / Output

NOTES

- This is a Mealy FSM. Output depends on present state and input.
- The sequence to be detected is 1101.
- When the machine is in state S3 and input X = 1, the sequence 1101 is detected and output Y = 1.
- After detection, the machine goes to state S1 to allow overlapping detection.

Meaning

State

S0	Initial state
S1	Received '1'
S2	Received '11'
S3	Received '110'

The sequence is detected when the machine is in state S3 and receives input 1.

Design Logic

The FSM continuously monitors the serial input.

Operation:

State S0

No match has occurred.

State S1

The first bit '1' has been detected.

State S2

The pattern '11' has been detected.

State S3

The pattern '110' has been detected.

If the next input is 1, the complete sequence 1101 is recognized and the output becomes HIGH.

The machine then transitions appropriately to support overlapping detection.

Simulation Results

Input sequence:

11011101

Output sequence:

00010001

The output pulse occurred twice, indicating successful detection of the sequence 1101 at both valid occurrences.

Observation

The sequence detector accurately recognized the target sequence within the serial input stream. The use of a Mealy architecture reduced the number of required states and enabled immediate output generation. The simulation results validated the theoretical operation of the detector.

Applications of FSMs

Finite State Machines are widely used in digital systems such as:

- Traffic light controllers
- Sequence detectors
- Communication protocols
- Vending machines
- Elevator controllers
- Washing machine controllers
- CPU control units
- Robotics
- Embedded systems
- Network packet processing

Their ability to model sequential behavior makes them fundamental components of digital design.

Overall Results

The Verilog HDL implementations of Moore FSM, Mealy FSM, Traffic Light Controller, and Sequence Detector were successfully simulated using Icarus Verilog and GTKWave. The generated waveforms and terminal outputs matched the expected theoretical behavior of each design.

The experiments demonstrated the practical implementation of sequential circuits and highlighted the differences between various FSM architectures.

Conclusion

The experiments provided a comprehensive understanding of finite state machines and their applications in digital system design. The Moore FSM demonstrated stable state-dependent outputs, whereas the Mealy FSM illustrated faster response through input-dependent outputs. The Traffic Light Controller showcased a real-world application of FSMs, and the Sequence Detector emphasized the importance of pattern recognition in digital communication systems.

The successful simulation of all four designs confirmed the correctness of the Verilog implementations and enhanced practical knowledge of sequential logic design using Verilog HDL, Icarus Verilog, and GTKWave.